

DEADLOCK GUARDIAN

Deadlock Risk Analizi, Tespiti ve Kurtarma Sistemi
İşletim Sistemleri Dersi — 11 Haftalık Proje Planı

Öğrenci Adı Soyadı	Öğrenci No	Projedeki Görevi
		Simülasyon ve veri modeli
		Deadlock tespiti ve recovery
		Risk analizi (Banker's Algorithm, risk seviyeleri)
		Arayüz ve görselleştirme

Ders Sorumlusu: _____ Teslim Tarihi: ____ / ____ / ____

1. Projenin Amacı

Bir işletim sisteminde birden fazla process aynı kaynakları (dosya, cihaz, kilit, bellek bölgesi) kullanmak ister. Bir process elindeki kaynağı bırakmadan başka bir process'in tuttuğu kaynağı beklerse ve bu bekleme karşılıklı hale gelirse hiçbir process ilerleyemez; bu duruma **deadlock (kilitlenme)** denir.

Deadlock Guardian, process ve kaynakları kontrollü bir simülasyon ortamında modelleyen bir yazılımdır. Sistem üç temel işi yerine getirir: **(1) Risk Analizi** — deadlock oluşmadan önce tehlikeli durumları belirleyip uyarır; **(2) Deadlock Tespiti** — kilitlenme oluştuğunda probleme dahil process ve kaynakları bulur; **(3) Recovery** — sistemi tekrar güvenli (SAFE) duruma getirmek için kurtarma seçenekleri sunar ve uygular. Amaç yalnızca bir algoritma yazmak değil, deadlock yönetimini kullanıcıya görsel ve anlaşılır biçimde sunan çalışan bir sistem geliştirmektir.

2. Kullanılacak İşletim Sistemi Konuları

Process ve process durumları · Kaynak tahsisi (resource allocation) · Deadlock için gerekli dört koşul (mutual exclusion, hold and wait, no preemption, circular wait) · Resource Allocation Graph · Cycle detection · Çok örnekli kaynaklar için deadlock detection algoritması · Güvenli durum ve Banker's Algorithm · Deadlock recovery (process sonlandırma, kaynak geri alma) · Senkronizasyon ve kilit mantığı. (Referans: Silberschatz, Galvin, Gagne — Operating System Concepts, 10. baskı, Bölüm 8: Deadlocks)

3. 11 Haftalık Geliştirme Planı — Özet

Aşağıdaki özet tablo haftaların sırasını, sonraki sayfalar ise her haftanın ayrıntılı iş listesini göstermektedir.

Hafta	Aşama	Haftanın Çıktısı
1	Konuyu Anlama ve Proje Tasarımı	Proje tanım belgesi ve kurulmuş kod deposu
2	Basit Simülasyon	Konsolda çalışan temel simülasyon (bu hafta deadlock tespiti yapılmaz)
3	Kaynak İlişkilerinin Takibi	Senaryo dosyasıyla çalışan sistem ve anlık durum tablosu
4	Deadlock Tespiti (Detection)	Deadlock'u tespit edip raporlayan modül ve testleri
5	Görselleştirme	Canlı güncellenen graf ekranı
6	Risk Analizi	Güvenli / güvensiz durum uyarıları
7	Risk Seviyesi Sistemi	Risk göstergesi ve açıklamalar
8	Recovery (Kurtarma)	Çalışan kurtarma modülü
9	Kullanıcı Arayüzü	Bütünleşik ve kullanılabilir arayüz
10	Test ve İyileştirme	Test raporu ve kararlı sürüm
11	Final Hazırlığı	Final demo, proje raporu ve sunum

4. Haftalık Ayrıntılı Plan

HAFTA 1 — KONUYU ANLAMA VE PROJE TASARIMI

Amaç: Deadlock konusunu kavramak ve projenin sınırlarını netleştirmek.

Yapılacaklar

- Deadlock, process, resource ve bekleme (waiting) kavramlarının incelenmesi
- Deadlock için gerekli dört koşulun öğrenilmesi: mutual exclusion, hold and wait, no preemption, circular wait
- Projenin modüllerinin belirlenmesi: simülasyon, tespit, risk analizi, kurtarma, arayüz
- Programlama dili ve araçların seçilmesi, ekip içi görev dağılımı
- GitHub deposunun ve proje klasör yapısının kurulması

İlgili konu: Deadlock kavramı, dört koşul (Coffman koşulları)

Haftanın çıktısı: Proje tanım belgesi ve kurulmuş kod deposu

HAFTA 2 — BASİT SİMÜLASYON

Amaç: Process ve kaynakların yazılımda modellendiği temel altyapıyı kurmak.

Yapılacaklar

- Process yapısının oluşturulması (kimlik, durum: RUNNING / WAITING / TERMINATED)
- Resource yapısının oluşturulması (kimlik, sahibi, bekleyen process kuyruğu)
- Kaynak isteme işlemi: kaynak boşsa process'e verilmesi
- Kaynak doluysa isteyen process'in WAITING durumuna alınıp kuyruğa eklenmesi
- Kaynak bırakıldığında sıradaki bekleyen process'e verilmesi
- Örnek: P1 → R1 alır, P2 → R1 ister ve bekler, P1 → R1'i bırakır, R1 → P2'ye geçer

İlgili konu: Process durumları, kaynak tahsisi, bekleme kuyrukları

Haftanın çıktısı: Konsolda çalışan temel simülasyon (bu hafta deadlock tespiti yapılmaz)

HAFTA 3 — KAYNAK İLİŞKİLERİNİN TAKIBI

Amaç: Sistemin herhangi bir andaki tam durumunu kayıt altında tutmak.

Yapılacaklar

- Hangi process'in hangi kaynağı tuttuğunu gösteren Allocation tablosunun tutulması
- Hangi process'in hangi kaynağı beklediğini gösteren Request tablosunun tutulması
- Birden fazla örneği olan kaynak desteği (örn. 3 adet yazıcı)
- Her işlemin zaman damgasıyla olay kaydına (log) yazılması
- Test senaryolarının JSON dosyasından yüklenebilmesi

İlgili konu: Available / Allocation / Request veri yapıları

Haftanın çıktısı: Senaryo dosyasıyla çalışan sistem ve anlık durum tablosu

HAFTA 4 — DEADLOCK TESPİTİ (DETECTION)

Amaç: Oluşan deadlock'u otomatik olarak bulmak.

Yapılacaklar

- Process ve kaynaklardan Resource Allocation Graph (RAG) oluşturulması
- DFS algoritması ile grafikte döngü (cycle) aranması
- Çok örnekli kaynaklar için deadlock detection algoritmasının uygulanması
- Deadlock'a dahil olan process ve kaynakların listelenmesi
- Doğru çalıştığını kanıtlayan birim testlerinin yazılması

İlgili konu: Resource Allocation Graph, cycle detection, detection algoritması

Haftanın çıktısı: Deadlock'u tespit edip raporlayan modül ve testleri

HAFTA 5 — GÖRSELLEŞTİRME

Amaç: Process-kaynak ilişkilerini kullanıcının bir bakışta anlayacağı şekilde göstermek.

Yapılacaklar

- Process'lerin (daire) ve kaynakların (kare) graf üzerinde çizilmesi
- Tutma (kaynak → process) ve bekleme (process → kaynak) kenarlarının ayrı gösterilmesi
- Deadlock döngüsünün kırmızı renkle vurgulanması
- Simülasyon ilerledikçe grafiğin güncellenmesi

İlgili konu: Resource Allocation Graph gösterimi

Haftanın çıktısı: Canlı güncellenen graf ekranı

HAFTA 6 — RISK ANALİZİ

Amaç: Deadlock oluşmadan önce tehlikeyi fark etmek (önleme yaklaşımı).

Yapılacaklar

- Güvenli durum (safe state) kavramının uygulanması
- Banker's Algorithm ile sistemin güvenli sırasının (safe sequence) hesaplanması
- Bir kaynak isteği karşılanmadan önce sistemin güvensiz duruma düşüp düşmeyeceğinin kontrolü
- Güvensiz durumda kullanıcıya uyarı verilmesi

İlgili konu: Safe state, Banker's Algorithm, deadlock avoidance

Haftanın çıktısı: Güvenli / güvensiz durum uyarıları

HAFTA 7 — RISK SEVİYESİ SİSTEMİ

Amaç: Riski derecelendirip nedenini açıklamak.

Yapılacaklar

- Risk puanlama kurallarının belirlenmesi (bekleme zincirleri, boş kaynak oranı, güvenli durum sonucu)
- Riskin Low, Medium, High ve Critical seviyelerine ayrılması
- Riskin neden yükseldiğinin açıklanması (örn. "P2 ve P4 karşılıklı kaynak bekliyor")
- Risk seviyesinin zaman içindeki değişiminin kaydedilmesi

İlgili konu: Kaynak kullanım analizi, güvensiz durum göstergeleri

Haftanın çıktısı: Risk göstergesi ve açıklamalar

HAFTA 8 — RECOVERY (KURTARMA)

Amaç: Oluşan deadlock'tan sonra sistemi tekrar güvenli duruma getirmek.

Yapılacaklar

- Yöntem 1 — Process sonlandırma: döngüdeki bir process'in sonlandırılıp kaynaklarının serbest bırakılması
- Yöntem 2 — Kaynak geri alma (preemption): bir process'ten kaynağın alınıp başkasına verilmesi
- Kurban seçimi: en düşük maliyetli process'in (öncelik, tuttuğu kaynak sayısı) belirlenmesi
- Kurtarma sonrası tespitin tekrar çalıştırılıp sistemin SAFE olduğunun doğrulanması

İlgili konu: Deadlock recovery, process termination, resource preemption

Haftanın çıktısı: Çalışan kurtarma modülü

HAFTA 9 — KULLANICI ARAYÜZÜ

Amaç: Tüm modülleri tek bir ekranda birleştirmek.

Yapılacaklar

- Dashboard: aktif process sayısı, kaynak sayısı, anlık risk seviyesi
- Process ve kaynak listeleri ile durumları
- Graf ekranı, deadlock uyarısı ve recovery seçenekleri
- Arayüzden process/kaynak ekleme, kaynak isteme ve bırakma

İlgili konu: Tüm modüllerin entegrasyonu

Haftanın çıktısı: Bütünleşik ve kullanılabilir arayüz

HAFTA 10 — TEST VE İYİLEŞTİRME

Amaç: Sistemin farklı durumlarda doğru ve kararlı çalıştığını göstermek.

Yapılacaklar

- Senaryo 1 — Normal çalışma: deadlock oluşmamalı
- Senaryo 2 — Tek deadlock: doğru tespit edilmeli
- Senaryo 3 — Birden fazla deadlock: hepsi ayrı ayrı bulunmalı
- Senaryo 4 — Recovery: sistem SAFE duruma dönmeli
- Bulunan hataların düzeltilmesi ve kodun düzenlenmesi

İlgili konu: Doğrulama ve test

Haftanın çıktısı: Test raporu ve kararlı sürüm

HAFTA 11 — FINAL HAZIRLIĞI

Amaç: Projeyi baştan sona çalışır halde sunmak.

Yapılacaklar

- Son hata kontrolleri
- Demo senaryosunun hazırlanması ve prova edilmesi
- Proje raporunun yazılması
- Sunumun hazırlanması

İlgili konu: Tüm konuların bütün halinde gösterimi

Haftanın çıktısı: Final demo, proje raporu ve sunum

5. Kullanılacak Teknolojiler

Katman	Teknoloji	Kullanım Amacı
Simülasyon ve algoritmalar	Python	Process/kaynak modeli, detection, Banker's ve recovery algoritmaları
Arayüz	PyQt veya web tabanlı arayüz	Dashboard, graf görselleştirme ve kullanıcı etkileşimi
Veri	JSON / SQLite	Senaryoların ve sonuçların saklanması
Ekip çalışması	Git + GitHub	Kod paylaşımı ve sürüm takibi

6. Final Demo Senaryosu

Sistem açılır → process ve kaynaklar oluşturulur → kaynaklar dağıtılır → karşılıklı bekleme senaryosu oluşturulur → sistem risk artışını gösterir → deadlock oluşur ve tespit edilir → probleme dahil process ve kaynaklar gösterilir → recovery seçenekleri sunulur → bir process sonlandırılıp kaynakları serbest bırakılır → sistem tekrar SAFE durumuna geçer.

7. Başarı Kriterleri

- Deadlock senaryolarının doğru tespit edilmesi ve dahil olan process/kaynakların gösterilmesi
- Deadlock oluşmadan önce risk uyarısı verilmesi
- En az bir recovery yönteminin uygulanması ve sistemin güvenli duruma döndüğünün gösterilmesi
- Normal ve deadlock durumlarının karşılaştırılabilmesi
- Kodun ekip üyelerinin bağımsız çalışabileceği şekilde modüler olması
- Finalde baştan sona çalışan bir demo sunulması